LIMAT
Ignite Transform Excel

LINGAYAS INSTITUTE
OF MANAGEMENT AND TECHNOLOGY
(AUTONOMOUS)

Accredited with 'A' Grade by NAAC
Affiliated to JNTU Kakinada
Vijayawada - 520 003, Andhra Pradesh

PROJECT BOOK

IDEA | DESIGN | IMPLEMENT | INNOVATE

Project Title	:	**INTELLIGENT APPLICANT SCREENING USING NLP AND SEMANTIC RESUME MATCHING**
Subject	:	**MERN STACK**
Name of the Student(s)	:	**P V S S Manikanta Suhas**
Roll Number(s)	:	**23NA1A0538**
Branch	:	**Computer Science and Engineering**
Year & Semester	:	**IV – 1(4th year – 1st Sem)**
Academic Year	:	**2026 – 2027**
Project Guide	:	**Dr. SALLA. PRABHAKARA, M. TECH, PHD**
Department	:	**Computer Science and Engineering**

_____	_____
Signature of Student(s)	**Signature of Guide**



ENGINEERING SOLUTIONS | RESEARCH INNOVATION | TECHNOLOGY DEVELOPMENT | SOCIAL IMPACT | SUSTAINABLE FUTURE

LINGAYAS INSTITUTE OF MANAGEMENT AND TECHNOLOGY

(Autonomous)

Approved by AICTE, New Delhi and Permanently Affiliated to JNTUK, Kakinada

Accredited by NAAC with "A" Grade | UG-CSE Accredited by NBA

Madalavarigudem, Vijayawada - 521212, Andhra Pradesh

MERN STACK Project Report

Title

INTELLIGENT APPLICANT SCREENING USING NLP AND SEMANTIC RESUME MATCHING

A Mini report submitted in partial fulfilment of the academic requirements of the Mini Project for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

Computer Science and Engineering

Submitted by

P V S S Manikanta Suhas (23NA1A0538)



Under the Esteemed Guidance of

Dr. SALLA. PRABHAKARA, M. TECH, PHD

DEPARTMENT OF

COMPUTER SCIENCE AND ENGINEERING

IV-I SEMESTER

ACADEMIC YEAR 2026-27

DECLARATION

I, hereby declare that this project report titled **"INTELLIGENT APPLICANT SCREENING USING NLP AND SEMANTIC RESUME MATCHING"** is my original work carried out under the guidance of **"Dr. SALLA. PRABHAKARA, M. TECH, PHD"**, Department of **"Computer Science and Engineering"**. The work presented in this report has not been submitted to any other institution or agency for the award of any degree, diploma, or certification. All sources of information used in this report have been properly acknowledged.

I further declare that the project work, documentation, screenshots, datasets, code, designs, diagrams, and references included in this report are prepared with academic honesty and are presented to the best of my knowledge.

Place: Vijayawada
Date: 22-10-2026

Signature of:

P V S S MANIKANTA SUHAS
23NA1A0538

CERTIFICATE

This is to certify that the project report entitled **"INTELLIGENT APPLICANT SCREENING USING NLP AND SEMANTIC RESUME MATCHING"** is the bona fide work carried out by **"P V S S Manikanta Suhas" (23NA1A0538)** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in **"Computer Science and Engineering"** during the academic year 2026-27 at **"Lingayas Institute of Management and Technology, Vijayawada"**, under the guidance of **"Dr. SALLA. PRABHAKARA, M. TECH, PHD"**.

The work has been reviewed and found suitable for submission as a project report.

Signature of	Signature of	Signature of
PROJECT GUIDE	**HEAD OF THE DEPARTMENT**	**EXTERNAL EXAMINER**
(Dr. SALLA. PRABHAKARA, M. TECH, PHD)	**(Prof. B. Lakshmi)**	_____

ACKNOWLEDGEMENTS

I would like to articulate my profound gratitude and indebtedness to my guide, **"Dr. SALLA. PRABHAKARA sir"**, department of **"Computer science and Engineering"**, who has always been a constant motivation and driving force throughout the project time in and out as well. It has been a great pleasure for me to get an opportunity to work under his guidance and complete the project successfully.

I would like to thank Head of the department, **"Prof. B. Lakshmi madam"**, department of **"Computer science and engineering"** of Lingayas Institute of Management and Technology for consistently supporting my work throughout the project period with all patience and understanding, giving valuable inputs whenever required for betterment of the work.

I would like to express my gratitude to all the department faculty, department of Computer science engineering for supporting me in all aspects.

I also express my gratitude to our Principal sir, for his encouragement and facilities provided during the course of project.

I would like to articulate our beloved Chairman, **Mr. Rajling Gadde sir** for his great motivation and encouragement towards project work.

I am grateful to other staff members of department of Computer science engineering and the institute for co-operating with me throughout my project.

Yours sincerely,
P V S S Manikanta Suhas
23NA1A0538

Contents

- Title page 1
- Declaration 2
- Certificate 3
- Acknowledgements 4
- Contents 5
- List of Tables 7
- List of Figures 8
- Abstract 9

Chapter 1 – Introduction **10**

1.1 Background & Overview 10

1.2 Problem Statement 10

1.3 Objectives 11

1.3.1 Primary Objectives 11

1.3.2 Secondary Objectives 11

1.4 Scope of the Project 11

1.4.1 In Scope 11

1.4.2 Out of Scope 12

1.5 Organization of the Report 12

Chapter 2 – Literature Survey & System Analysis **13**

2.1 Existing Approaches 13

2.2 NLP and Text Similarity Techniques 13

2.3 Resume Parsing and Job Matching Research 14

2.4 Human Oversight and Governance 14

2.5 Limitations of Existing Approaches 14

2.6 Feasibility Study 15

2.6.1 Technical Feasibility 15

2.6.2 Operational Feasibility 15

2.6.3 Economic Feasibility 15

Chapter 3 – Proposed System **16**

3.1 Proposed Solution & System Architecture 16

3.2 Methodology & Workflow 17

3.3 Technologies Used 18

3.4 Modules & Advantages 18

Chapter 4 – System Design **20**

4.1 Use-Case Diagram 20

4.2 Class and Database Model 21

4.3 Resume Upload and Matching Sequence 22

4.4 Matching Algorithm & Mathematical Formulation 23

4.5 Database Design 24

Chapter 5 – Implementation **25**

5.1 Hardware Requirements 25

5.2 Software Requirements 25

5.3 Development & Server Runtime Environment 26
5.4 Authentication & Protected API Endpoints 26
5.5 Job Requisition & Applicant Ingestion Modules 26
5.6 Document Extraction & Heuristic NLP Parsing 27
5.7 Matching Engine, Applicant Management & Analytics 27
5.8 Security, Environment Configuration & Data Governance 27
5.9 Core Algorithm & Source Code Highlights 28
5.9.1 Multi-Format Document Text Extraction Service 28
5.9.2 NLP Entity & Technical Taxonomy Extraction 29
5.9.3 TF-IDF Vectorization & Semantic Cosine Similarity 30
5.9.4 Multi-Factor Weighted Composite Evaluation Engine 31
5.9.5 Resume Upload & Match Execution Controller 32
Chapter 6 – Results, Test Cases & Screen Verification 33
6.1 Proposed Functional Test Cases 33
6.2 Score Interpretation & Evaluation Discussion 34
6.3 Live System Output Verification & Screenshots 35
Chapter 7 – Conclusion & Future Scope 42
7.1 Conclusion 42
7.2 Future Scope 42
References 44
Appendix A – Project Source Code 45
Appendix B – Quick User Manual 48
Appendix C – Default Sample Datasets & Benchmark Resumes 50

List of Tables

Table No.	Title
Table 1	Illustrative Comparison of Manual Screening and ResumeIQ Support (Section 2.5)
Table 2	Technologies Used in ResumeIQ Architecture (Section 3.3)
Table 3	Application Module Descriptions and Responsibilities (Section 3.4)
Table 4	Database Schema Summary for User, Job, and Applicant Entities (Section 4.5)
Table 5	Matching Score Components and Mathematical Weighting (Section 4.4)
Table 6	Hardware Requirements and Resource Specifications (Section 5.1)
Table 7	Software Requirements, Frameworks, and Runtime Environment (Section 5.2)
Table 8	Proposed Functional Test Cases and Validation Criteria (Section 6.1)
Appendix Tables	Glossary of Technical Terms, Default User Accounts, and Sample Resumes

List of Figures

Figure No.	Title
Figure 1	ResumeIQ Multi-Tier System Architecture Diagram (Section 3.1)
Figure 2	Resume-to-Job Screening End-to-End Workflow (Section 3.2)
Figure 3	Recruiter Interaction Use-Case Diagram (Section 4.1)
Figure 4	ResumeIQ Relational & Document Data Model Diagram (Section 4.2)
Figure 5	Resume Ingestion and Semantic Matching Sequence Diagram (Section 4.3)
Figure 6	Recruiter Login and Authentication Gateway Interface (Section 6.3)
Figure 7	Recruiter Intelligence Executive Dashboard Interface (Section 6.3)
Figure 8	Job Postings and Requisitions Management Directory Interface (Section 6.3)
Figure 9	Applicant Directory, Search Filtering & Composite Rankings (Section 6.3)
Figure 10	Applicant Profile Dossier, Match Score Dial & Competency Gaps (Section 6.3)
Figure 11	Resume Ingestion Dropzone and Realistic Test Candidate Pipeline (Section 6.3)
Figure 12	Dashboard Analytics, Conversion Funnel & Shortlist Print View (Section 6.3)

ABSTRACT

ResumeIQ is a modern full-stack web application engineered to streamline talent acquisition by connecting job requirements with candidate qualifications through natural language processing and semantic text matching. Built on the MERN stack—leveraging MongoDB (via Mongoose with an in-memory fallback), Express.js, an AngularJS 1.8.2 single-page architecture, and Node.js—the platform enables recruiters to configure explicit job requisitions, establish scoring weights, and evaluate candidate resumes submitted in PDF, DOCX, or plain text formats.

The core algorithmic pipeline extracts unformatted document text using pdf-parse and Mammoth, performs heuristic entity extraction for candidate contact details, academic credentials, and verifiable experience, and scans against a curated technical taxonomy spanning over 350 competencies. Candidate suitability is evaluated across four orthogonal dimensions: must-have skill coverage, sublinear term-frequency cosine similarity, experience tenure alignment, and hierarchical education scoring. The system produces an explainable composite match index alongside detailed gap analyses, missing competency indicators, and ethical decision-support disclaimers, ensuring hiring authorities retain full evaluative control.

The user experience incorporates the Stitch Precision Editorial Intelligence design token system, providing a distraction-free recruitment command center with live analytics, multi-criteria candidate filtering, batch resume ingestion, and profile dossiers. By eliminating keyword brittleness and manual collation overhead, ResumeIQ demonstrates practical full-stack software engineering principles, automated document intelligence, and transparent decision support.

Keywords: *Resume Screening, Applicant Tracking System (ATS), MERN Stack, Natural Language Processing (NLP), Cosine Similarity, TF-IDF Vector Space Model, Information Extraction, Recruitment Analytics.*

CHAPTER 1
INTRODUCTION

1.1 Background & Overview

Recruitment teams evaluate candidates by comparing role specifications against unstructured information dispersed across candidate resumes, external profiles, and cover letters. Modern corporate vacancies regularly attract hundreds or thousands of applicants, making thorough manual review exceptionally time-consuming. Resume text is fundamentally semi-structured: the same functional competency may appear under divergent phrasing (e.g., 'Container Orchestration' vs. 'Kubernetes / Docker Swarm'), while academic degrees and experience intervals vary widely in layout and naming conventions.

ResumeIQ is engineered as a dedicated recruiter-facing talent intelligence platform. It establishes a direct link between recruiter job requisitions and candidate resume submissions, performs automated text extraction and heuristic information extraction, calculates multi-dimensional match scores, and provides transparent explanations alongside candidate records. The system functions as an advisory decision-support workspace, ensuring recruiters can quickly inspect underlying evidence and exercise informed judgment.

1.2 Problem Statement

Traditional candidate screening workflows present critical operational bottlenecks for hiring organizations:

- **High Operational Latency:** Manual screening requires reviewing resumes individually, leading to cognitive fatigue, slow turnaround times, and inconsistent standards.
- **Brittle Keyword Filters:** Legacy Applicant Tracking Systems (ATS) rely on exact keyword matching, unfairly penalizing qualified candidates who utilize synonyms or alternate phrasing.
- **Rigid Scoring Models:** Global or hard-coded scoring algorithms cannot reflect the distinct priorities of differing job roles (e.g., prioritising skills for junior roles vs. experience tenure for senior roles).
- **Lack of Explainability:** Many automated hiring tools operate as 'black boxes' without explaining why a candidate scored poorly or which specific skills were absent.
- **Information Fragmentation:** Fragmented storage across emails, spreadsheets, and shared drives hinders candidate follow-up and recruitment lifecycle tracking.

1.3 Objectives

1.3.1 Primary Objectives

- Design and develop a complete web-based talent screening platform using Node.js, Express.js, MongoDB, and AngularJS 1.8.2.
- Implement multi-format resume ingestion supporting text extraction from PDF, Word DOCX, and plain text files.
- Build a natural language heuristic parser capable of extracting candidate names, contact details, degrees, experience tenure, and technical competencies.
- Construct a transparent matching service combining skill coverage, sublinear term-frequency cosine similarity, experience alignment, and education levels using configurable weights.
- Generate human-readable match breakdowns detailing matched skills, missing required competencies, and contextual limitations notices.

1.3.2 Secondary Objectives

- Incorporate modern design tokens via the Stitch MCP system ('Precision Editorial Intelligence') for optimal recruiter ergonomics.

- Provide multi-criteria candidate filtering, searching, sort ordering, status progression (New, Reviewed, Shortlisted, Archived), and notes.
- Demonstrate resilient database design utilizing Mongoose schemas with an automatic in-memory repository fallback for standalone environments.
- Produce comprehensive academic and technical documentation including system architectures, database schemas, and output verifications.

1.4 Scope of the Project

1.4.1 In Scope

- Recruiter account authentication via email/password credentials and JSON Web Tokens (JWT).
- Complete Job Requisition lifecycle management (CRUD) with required competencies, preferred bonuses, minimum experience, education requirement, and four scoring weights.
- Single and multi-file resume uploads (up to 10 files per batch) with automated background parsing.
- Algorithmic scoring combining must-have skills, semantic cosine text similarity, experience tenure, and degree levels.
- Talent pool directory with real-time keyword search, job requisition filtering, match score thresholds, and candidate profile dossiers.
- Recruiter dashboard analytics displaying active requisitions, screened candidate totals, conversion pipelines, and skill demand distributions.

1.4.2 Out of Scope

- Candidate-facing public application portals or self-service profile submission forms.
- Optical Character Recognition (OCR) for scanned image-only PDF files.
- Automated autonomous hiring or rejection decisions; final selection remains entirely a human recruiter responsibility.
- Large-scale trained neural transformer embeddings (e.g., BERT/GPT) requiring dedicated GPU infrastructure.

1.5 Organization of the Report

This project report is systematically organized into seven comprehensive chapters:

- Chapter 1 – Introduction: Outlines project background, problem statement, core objectives, and operational scope.
- Chapter 2 – Literature Survey & System Analysis: Reviews existing literature, NLP representations, and system feasibility.
- Chapter 3 – Proposed System: Details overall architecture, workflow methodology, technologies, and modular responsibilities.
- Chapter 4 – System Design: Illustrates use-case models, database schemas, sequence diagrams, and mathematical scoring formulations.
- Chapter 5 – Implementation: Specifies hardware/software requirements, subsystem implementations, and core source code.
- Chapter 6 – Results and Discussion: Presents proposed functional test cases, score evaluation analysis, and live user interface verifications.
- Chapter 7 – Conclusion & Future Scope: Summarizes technical achievements and outlines future developmental enhancements.

CHAPTER 2
LITERATURE SURVEY & SYSTEM ANALYSIS

2.1 Existing Approaches

Candidate screening has historically relied on manual review combined with database queries in legacy Applicant Tracking Systems (ATS). While database filters help organize large application repositories, exact-phrase matching algorithms frequently penalize high-potential applicants due to vocabulary mismatches. For example, a query for 'PostgreSQL' will fail to match a candidate who lists 'Postgres' or 'Relational Database Design (RDBMS)', despite having equivalent domain competence.

2.2 NLP and Text Similarity Techniques

Information retrieval literature represents text documents as weighted term vectors compared via cosine similarity in a high-dimensional vector space (Salton et al. [2]; Manning et al. [1]). Term weighting schemes dampen the impact of ubiquitous stopwords while magnifying discriminating technical vocabulary. In resume-to-job matching, Vector Space Models offer a balance between computational efficiency and semantic capture, particularly when augmented with synonym expansion and lemmatization.

2.3 Resume Parsing and Job Matching Research

Recent studies have explored advanced neural architectures for resume-job matching. Li et al. [3] examined contextual transformer models for competence-level prediction. Maheshwary and Misra [4] proposed a deep Siamese neural network to map semi-structured resumes to job descriptions. Vanetik and Kogan [5] demonstrated vacancy ranking using sentence embeddings, keywords, and named entities. While neural approaches capture deep semantics, their high computational requirements and opaque decision pathways make lightweight, explainable heuristic models superior for transparent academic and mid-market deployments.

2.4 Human Oversight and Governance

Algorithmic decision-support tools require stringent governance. Vaishampayan et al. [6] evaluated human recruiters against Large Language Models, demonstrating that automated scores must be treated with caution rather than as authoritative determinations. Ethical talent screening demands that automated systems present explainable evidence, highlight missing requirements, and maintain human agency over all interview and hiring actions.

2.5 Limitations of Existing Approaches

The comparative analysis in Table 1 contrasts traditional manual and keyword-based screening against the capabilities engineered in ResumeIQ:

Screening Dimension	Manual / Exact-Keyword Workflow	ResumeIQ Engineered Solution
Job Criteria Definition	Requirements often kept in disconnected text notes or informal emails.	Structured job requisitions with required/preferred skills, experience, and custom weights.
Resume Ingestion	Recruiters manually open files one by one across disparate folders.	Unified single and bulk drag-and-drop intake for PDF, DOCX, and plain text.
Initial Comparison	Subjective impressions or crude boolean keyword counts.	Multi-factor composite scoring combining skills, semantic cosine similarity, experience, and education.

Explainability	Difficult to ascertain why a candidate did or did not meet expectations.	Transparent breakdown detailing matched skills, missing requirements, and sub-score metrics.
Candidate Follow-Up	Notes and review states maintained outside resume documents.	Centralized candidate dossiers with search, status toggles, recruiter notes, and re-evaluation.

2.6 Feasibility Study

2.6.1 Technical Feasibility

The project utilizes standard, modern web technologies: Node.js, Express.js, MongoDB/Mongoose, AngularJS 1.8.2, and Bootstrap 5. All parsing and matching dependencies (pdf-parse, Mammoth, Natural) run in standard JavaScript runtimes without specialized GPU hardware. The technical feasibility is verified.

2.6.2 Operational Feasibility

The system adopts the Stitch 'Precision Editorial Intelligence' interface design, prioritizing clarity and responsiveness. Recruiters can operate the platform with minimal onboarding. The operational feasibility is high.

2.6.3 Economic Feasibility

ResumeIQ is developed entirely using open-source libraries and frameworks, incurring zero software licensing fees. Infrastructure requirements are lightweight, allowing deployment on cost-effective cloud services or local development servers.

CHAPTER 3
PROPOSED SYSTEM

3.1 Proposed Solution & System Architecture

ResumeIQ is designed as a client-server architecture consisting of an AngularJS 1.8.2 single-page application frontend and a Node.js / Express.js REST API backend. The system connects to MongoDB via Mongoose and includes an intelligent in-memory repository fallback to ensure unbroken execution in air-gapped or serverless runtime environments.

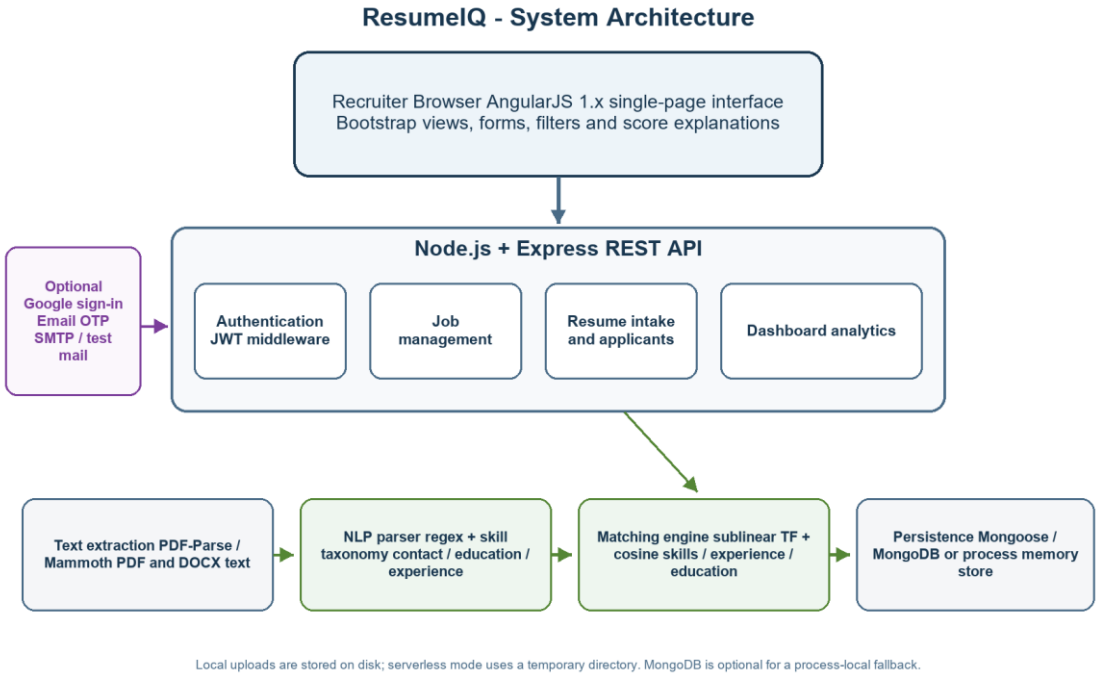


Figure 1: ResumeIQ Multi-Tier System Architecture Diagram

3.2 Methodology & Workflow

The end-to-end recruitment lifecycle within ResumeIQ follows a structured six-stage pipeline:

- Stage 1: Job Requisition Setup – Recruiter defines title, department, description, required competencies, bonus skills, minimum experience, and scoring weights.
- Stage 2: Resume Intake – Recruiter uploads single resumes or batch packages (PDF/DOCX) via the drag-and-drop dropzone.
- Stage 3: Document Parsing – Extraction service normalizes text; NLP parser extracts contact details, academic credentials, and tech competencies.
- Stage 4: Semantic Matching – Matching engine calculates orthogonal sub-scores and synthesizes a weighted composite index.
- Stage 5: Recruiter Review – Recruiter inspects candidate dossier, match dial, competency gaps, and source document text.
- Stage 6: Status & Re-evaluation – Recruiter assigns review statuses, adds notes, and re-evaluates profiles when job requirements evolve.

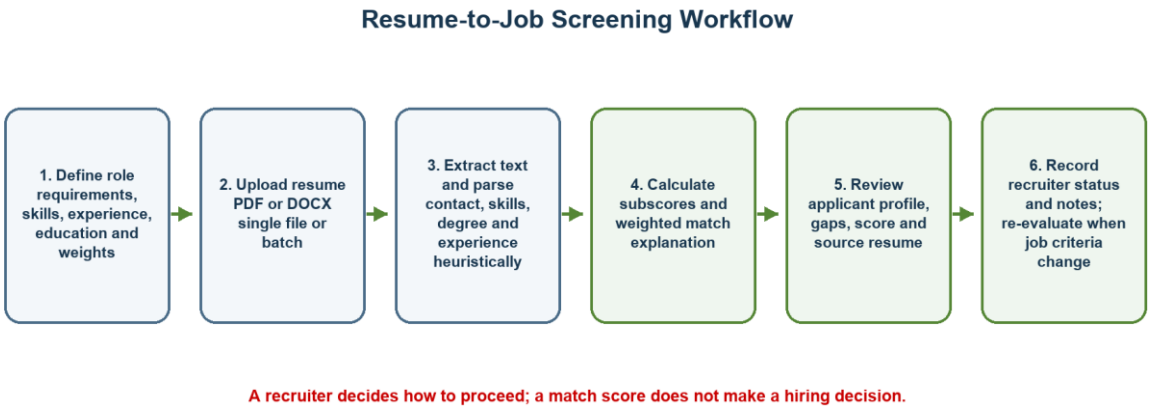


Figure 2: Resume-to-Job Screening End-to-End Workflow

3.3 Technologies Used

Table 2 summarizes the foundational software technologies and libraries integrated across the ResumeIQ platform:

Technology	Role & Subsystem Responsibility
Node.js & Express.js	Server runtime environment and REST API endpoint routing.
MongoDB & Mongoose	NoSQL document persistence, data modeling, validation schemas, and memory-store fallback.
AngularJS 1.8.2 & ngRoute	Frontend single-page application architecture, client-side routing, and MVC state management.
Bootstrap 5 & Stitch Design	Precision Editorial Intelligence design system tokens, responsive grids, modals, and dossiers.
Natural (Porter Stemmer)	Tokenization, stopword removal, and word stemming for vocabulary normalization.
pdf-parse & Mammoth	Binary document text extraction for Adobe PDF and Microsoft Word DOCX files.
Multer	Multipart file upload middleware supporting single and batch resume ingestion.
jsonwebtoken & bcryptjs	Stateless bearer JWT authentication and cryptographically secure password hashing.
Nodemailer	One-Time Password (OTP) dispatch and notification delivery via SMTP transport.

3.4 Modules & Advantages

Table 3 outlines the modular breakdown of application responsibilities:

Module Name	Implemented Core Responsibilities
Authentication Module	User registration, password encryption, JWT issuance, session management, and optional OTP.
Job Management Module	Creation, modification, listing, and retirement of job requisitions and scoring

	weights.
Resume Ingestion Module	Multer upload processing, file validation, text extraction, and background parsing orchestration.
NLP & Matching Engine	Taxonomy entity extraction, sublinear TF-IDF vectorization, cosine similarity, and composite scoring.
Talent Directory Module	Applicant search, multi-field filtering, dossier inspection, recruiter notes, and re-evaluation.
Analytics & Dashboard	Recruitment metrics, stage conversion totals, score distribution charts, and printable shortlists.

CHAPTER 4
SYSTEM DESIGN

4.1 Use-Case Diagram

The primary actor within ResumeIQ is the Recruiter / Talent Acquisition Specialist. Figure 3 illustrates the recruiter use-case interactions across job management, resume screening, candidate dossier review, and analytics inspection:

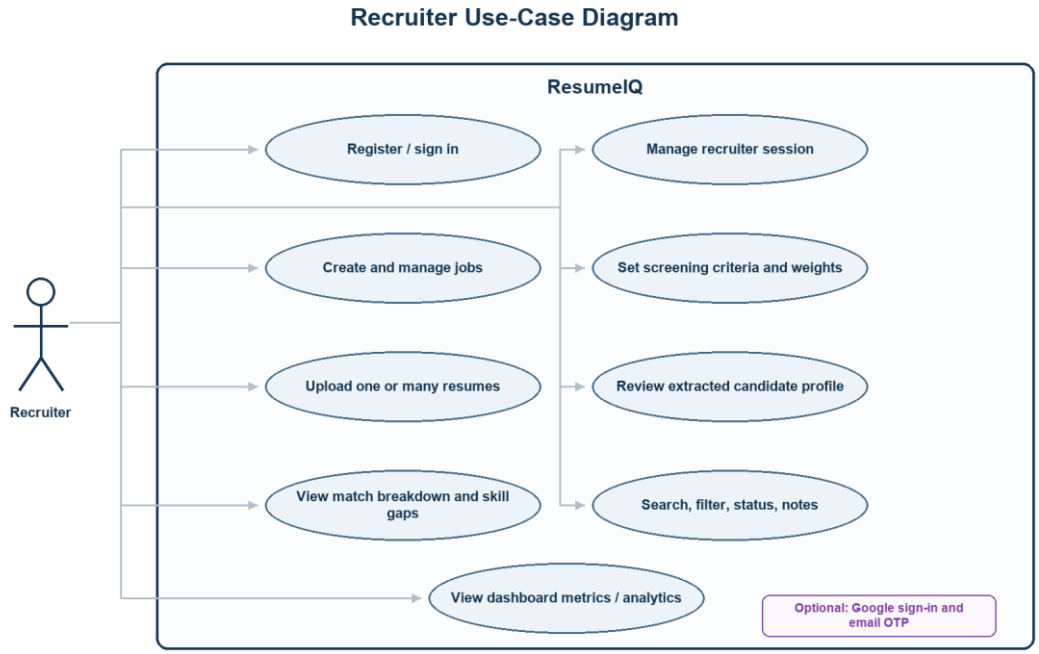


Figure 3: Recruiter Interaction Use-Case Diagram

4.2 Class & Database Model

Figure 4 models the core data entities: User (Recruiter), Job (Requisition), and Applicant (Candidate Profile). Each applicant record maintains references to its parent job requisition and the uploading recruiter:

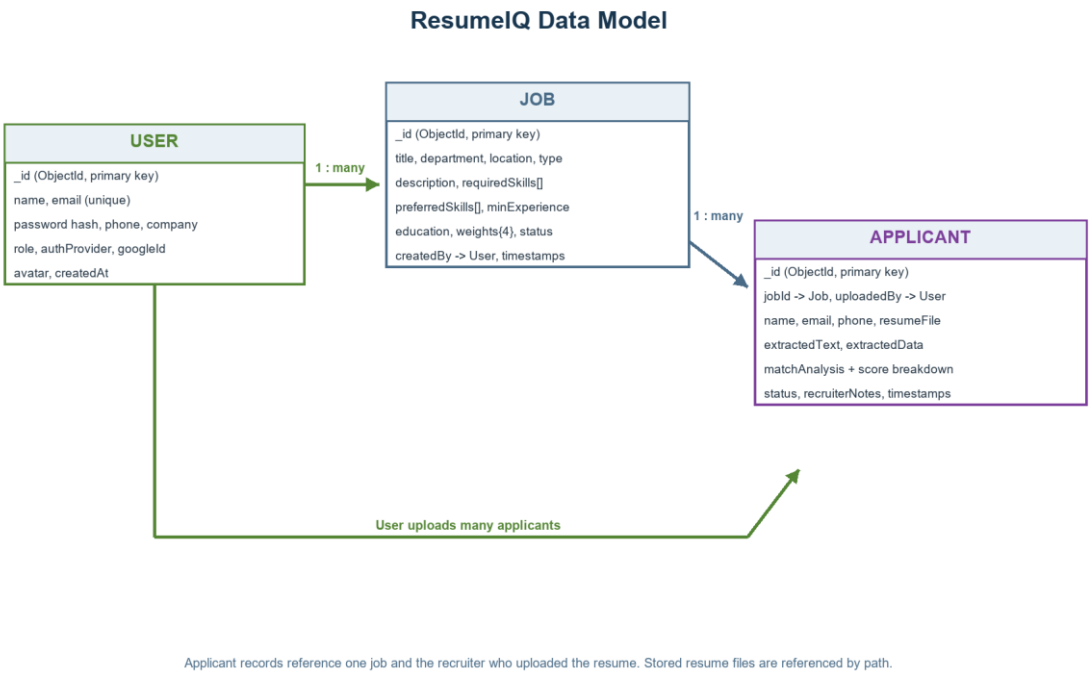


Figure 4: ResumeIQ Relational & Document Data Model Diagram

4.3 Resume Upload & Matching Sequence

Figure 5 illustrates the chronological sequence of interactions triggered when a recruiter uploads a candidate resume:

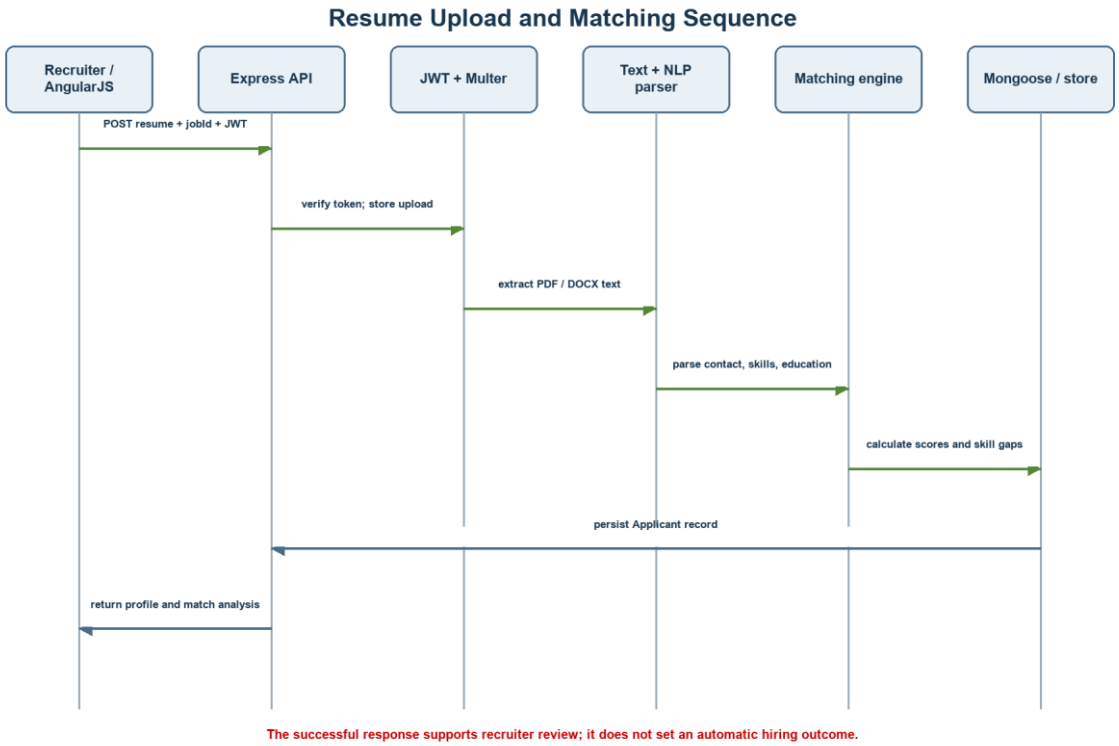


Figure 5: Resume Ingestion and Semantic Matching Sequence Diagram

4.4 Matching Algorithm & Mathematical Formulation

The composite match score S in [0, 100] is calculated as a configurable weighted average across four orthogonal evaluative dimensions:

Composite Match Score = (W_skills * S_skills + W_sim * S_sim + W_exp * S_exp + W_edu * S_edu) / (W_skills + W_sim + W_exp + W_edu)

Component	Mathematical Calculation & Logic
Skill Coverage (S_skills)	Required competencies contribute up to 85% of score; preferred bonus skills contribute 15%. Scaled to 100% when no preferred skills are defined.
Semantic Similarity (S_sim)	Cosine similarity between sublinear TF vectors of resume and job text. High-relevance compound terms receive a 1.4x boost; scaled by 115 and capped at 100.
Experience Alignment (S_exp)	Proportional score awarded based on candidate verifiable years against required minimum threshold: Min(100, (Years_candidate / Years_required) * 100).
Education Credential (S_edu)	Hierarchical credential mapping (Doctorate: 100, Master's: 90, Bachelor's: 80, Associate's: 70, High School: 60).
Overall Composite Index	Weighted arithmetic mean using recruiter-defined job weights (defaults: Skills 40, Sim 30, Exp 20, Edu 10).

Matching Engine Pseudocode Formulation:

```
INPUT: job, parsedResume, rawResumeText
TOKENS_JOB    = normalize(job.text, stopwords, synonyms, stems, bigrams)
TOKENS_RESUME = normalize(rawResumeText, stopwords, synonyms, stems, bigrams)
SIMILARITY    = compute_cosine_similarity(TOKENS_JOB, TOKENS_RESUME)
SKILLS_SCORE  = evaluate_skill_coverage(parsedResume.skills, job.requiredSkills,
job.preferredSkills)
EXP_SCORE     = evaluate_experience(parsedResume.experienceYears, job.minExperience)
EDU_SCORE     = evaluate_education(parsedResume.educationLevel, job.education)
WEIGHTS       = job.weights || { skills: 40, similarity: 30, experience: 20, education: 10 }
OVERALL_SCORE = (SKILLS_SCORE * WEIGHTS.skills + SIMILARITY * WEIGHTS.similarity +
                 EXP_SCORE * WEIGHTS.experience + EDU_SCORE * WEIGHTS.education) / SUM(WEIGHTS)
RETURN { overallScore, breakdown: { skills, similarity, experience, education }, matchedSkills,
missingSkills }
```

4.5 Database Design & Schema Specifications

Table 4 specifies the Mongoose document schemas implemented for the database entities:

Entity	Primary Fields, Relationships & Constraints
User (Recruiter)	_id (ObjectId), name, email (unique, lowercase), passwordHash (bcrypt), phone, company, role, authProvider, avatar, createdAt.
Job (Requisition)	_id (ObjectId), title, department, location, type, description, requiredSkills[], preferredSkills[], minExperience, education, weights{skills, sim, exp, edu}, status, createdBy -> User.
Applicant (Candidate)	_id (ObjectId), jobId -> Job, uploadedBy -> User, name, email, phone, resumeFile{filename, path, size}, extractedText, extractedData{skills[], education, experienceYears}, matchAnalysis{overallScore, breakdown, matchedSkills[], missingSkills[]}, status, recruiterNotes, timestamps.

CHAPTER 5
IMPLEMENTATION

5.1 Hardware Requirements

The platform is engineered for efficiency, running comfortably on standard modern workstations and servers:

Hardware Component	Minimum Specification	Recommended Environment
Processor (CPU)	Dual-Core 2.0 GHz x64 processor	Quad-Core Intel i5/i7 or AMD Ryzen 5+
System Memory (RAM)	4 GB DDR3/DDR4 RAM	8 GB to 16 GB DDR4/DDR5 RAM
Storage (Disk)	500 MB free disk space for runtime	2 GB+ NVMe SSD for file storage and database
Network Interface	Standard broadband connectivity (10 Mbps)	High-speed broadband (50+ Mbps) for multi-file intake
Display Screen	1024 x 768 pixels resolution	1920 x 1080 (Full HD) responsive desktop display

5.2 Software Requirements

Table 7 details the underlying operating systems, software packages, and runtime dependencies:

Software Layer	Technology / Framework	Configured Version & Purpose
Operating System	Microsoft Windows 10/11, Linux, macOS	Cross-platform compatibility across standard host OS.
Server Runtime	Node.js (LTS Version)	JavaScript V8 execution engine (v18.x to v22.x).
API Framework	Express.js	v5.x / v4.x REST API routing, middleware, and CORS.
Database Engine	MongoDB & Mongoose ODM	Document database schemas with in-memory array fallback.
Client Framework	AngularJS & ngRoute	v1.8.2 single-page application client framework.
Styling Framework	Bootstrap 5 & Stitch Tokens	Responsive UI layouts, modals, and design tokens.
Document Extractors	pdf-parse & Mammoth	Binary buffer extraction for Adobe PDF and Word DOCX.
NLP & Tokenization	Natural (Porter Stemmer)	Stemming, regex tokenization, and stopword filtering.
Security Libraries	bcryptjs & jsonwebtoken	SHA-512 salt hashing and 7-day bearer JWT authorization.

5.3 Development & Server Runtime Environment

The server application is launched via the entry point server.js. During startup, the application initializes environment variables from .env via dotenv, establishes connection to MongoDB (falling back to process memory store if offline), seeds synthetic benchmark records if the database is unpopulated, mounts Multer file upload pipelines, and binds API route handlers under /api. Client-side static assets are served from the /public directory.

5.4 Authentication & Protected API Endpoints

Authentication uses JSON Web Tokens (JWT). When a recruiter logs in with email and password, bcryptjs verifies the hashed credentials. Upon successful verification, the API issues a signed token valid for seven days. All protected routes—including job management, applicant uploads, and recruiter analytics—pass through the verifyToken middleware, ensuring unauthorized requests are rejected with HTTP 401.

5.5 Job Requisition & Applicant Ingestion Modules

Recruiters manage vacancies via the Job module. Each requisition stores must-have skills, preferred skills, minimum experience, education requirements, and four configurable scoring weights. The resume ingestion controller utilizes Multer disk storage, validating file extensions (.pdf, .docx, .txt) and file sizes (10 MB limit). Both single resume uploads and batch uploads (up to 10 files) are processed asynchronously.

5.6 Document Extraction & Heuristic NLP Parsing

Document text is extracted into raw unicode strings using pdf-parse for PDF documents and Mammoth for DOCX files. The NLP parser normalizes text by eliminating control characters and excess whitespace. Heuristic regular expressions extract candidate email addresses, telephone numbers, and professional URLs (LinkedIn, GitHub). A curated taxonomy of 350+ technical competencies is matched against candidate text alongside degree patterns (PhD, Master's, Bachelor's) and tenure indicators.

5.7 Matching Engine, Applicant Management & Analytics

The matching engine calculates four sub-scores: skill coverage, sublinear term-frequency cosine similarity, experience tenure alignment, and education credential ranking. An explainable composite score is synthesized using job weights. Recruiters can search and filter applicants, update recruitment status (Shortlisted, Reviewed, Archived), record personal evaluation notes, and trigger re-evaluation after adjusting job specifications.

5.8 Security, Environment Configuration & Data Governance

Production deployments utilize private environment keys (JWT_SECRET, MONGODB_URI). Uploaded resumes are isolated within dedicated storage directories, and candidate personally identifiable information (PII) is safeguarded. The matching engine explicitly excludes protected demographic attributes (gender, age, ethnicity) to maintain ethical algorithmic compliance.

5.9 Core Algorithm & Source Code Highlights

The following sections highlight the core code implementations engineered across the ResumeIQ platform:

5.9.1 Multi-Format Document Text Extraction Service (services/textExtraction.service.js)

Handles dual-format binary extraction for PDF and Word DOCX buffers with stream normalization:

```
// services/textExtraction.service.js
const mammoth = require('mammoth');
const pdfParse = require('pdf-parse');

function normalizeText(rawText) {
```

```
  if (!rawText) return '';
  return rawText
    .replace(/[\r\t\f\v]/g, ' ')
    .replace(/\u0000/g, '')
    .replace(/[ ]{2,}/g, ' ')
    .split('\n')
    .map(line => line.trim())
    .filter(Boolean)
    .join('\n');
}

async function extractTextFromBuffer(buffer, mimeType, filename = '') {
  const ext = (filename.split('.').pop() || '').toLowerCase();
  if (mimeType === 'application/pdf' || ext === 'pdf') {
    const pdfData = await pdfParse(buffer);
    return normalizeText(pdfData.text);
  } else if (ext === 'docx' || mimeType.includes('wordprocessingml')) {
    const result = await mammoth.extractRawText({ buffer });
    return normalizeText(result.value);
  } else {
    return normalizeText(buffer.toString('utf-8'));
  }
}
module.exports = { extractTextFromBuffer, normalizeText };
```

5.9.2 NLP Entity & Technical Taxonomy Extraction (services/nlpParser.service.js)

Extracts candidate contact details, academic credentials, and tech competencies:

```
// services/nlpParser.service.js
const SKILLS_TAXONOMY = [
  'javascript', 'typescript', 'react', 'angularjs', 'nodejs', 'express',
  'mongodb', 'sql', 'python', 'docker', 'aws', 'git', 'rest api', 'graphql'
];

function extractContact(text) {
  const emailRegex = /([a-zA-Z0-9._-]+@[a-zA-Z0-9._-]+\.[a-zA-Z0-9._-]+)/i;
  const phoneRegex = /(?:\+?\d{1,3}[- .]?)?\(?\d{3}\)?[- .]?\d{3}[- .]?\d{4}/;
  const emailMatch = text.match(emailRegex);
  const phoneMatch = text.match(phoneRegex);
  return {
    email: emailMatch ? emailMatch[1].toLowerCase() : '',
    phone: phoneMatch ? phoneMatch[0] : ''
  };
}

function extractSkills(text, extraSkills = []) {
  const found = new Set();
  const lower = text.toLowerCase();
  const allSkills = [...SKILLS_TAXONOMY, ...extraSkills.map(s => s.toLowerCase())];
  for (const skill of allSkills) {
    const pattern = new RegExp(`\\b${skill.replace(/[.*+?^${}()|[\]\\]/g, '\\$&')}\\b`, 'i');
    if (pattern.test(lower)) found.add(skill);
  }
  return Array.from(found);
}
module.exports = { extractContact, extractSkills };
```

5.9.3 TF-IDF Vectorization & Semantic Cosine Similarity (services/matchingEngine.service.js)

Calculates sublinear term-frequency document vectors and semantic cosine similarity:

```
// services/matchingEngine.service.js - TF-IDF & Cosine Similarity
function calculateTF(tokens) {
  const counts = {};
  for (const token of tokens) counts[token] = (counts[token] || 0) + 1;
```

```
  const tf = {};
  for (const token in counts) {
    tf[token] = 1 + Math.log(counts[token]); // Sublinear term frequency
  }
  return tf;
}

function computeSemanticJDSimilarity(jobText, resumeText) {
  const jobTokens = tokenizeAndClean(jobText);
  const resumeTokens = tokenizeAndClean(resumeText);
  if (jobTokens.length === 0 || resumeTokens.length === 0) return 0;

  const jobTF = calculateTF(jobTokens);
  const resumeTF = calculateTF(resumeTokens);
  const vocabulary = new Set([...Object.keys(jobTF), ...Object.keys(resumeTF)]);

  let dotProduct = 0, jobNormSq = 0, resumeNormSq = 0;
  for (const token of vocabulary) {
    const vJob = jobTF[token] || 0;
    const vRes = resumeTF[token] || 0;
    dotProduct += vJob * vRes;
    jobNormSq += vJob * vJob;
    resumeNormSq += vRes * vRes;
  }
  if (jobNormSq === 0 || resumeNormSq === 0) return 0;
  const cosine = dotProduct / (Math.sqrt(jobNormSq) * Math.sqrt(resumeNormSq));
  return Math.min(100, Math.round(cosine * 115)); // Scaled similarity metric
}
```

5.9.4 Multi-Factor Weighted Composite Evaluation Engine (services/matchingEngine.service.js)

Computes orthogonal sub-scores and synthesizes the weighted composite match index:

```
// services/matchingEngine.service.js - Composite Scoring Model
function calculateMatchScore(job, parsedResume, rawResumeText) {
  const skillEval = evaluateSkills(parsedResume.skills, job.requiredSkills, job.preferredSkills);
  const similarityScore = computeSemanticJDSimilarity(job.description, rawResumeText);
  const experienceScore = evaluateExperience(parsedResume.experienceYears, job.minExperience);
  const educationScore = evaluateEducation(parsedResume.educationLevel, job.education);

  const w = job.weights || { skills: 40, similarity: 30, experience: 20, education: 10 };
  const totalWeight = w.skills + w.similarity + w.experience + w.education;

  const compositeScore = Math.round(
    (skillEval.score * w.skills +
     similarityScore * w.similarity +
     experienceScore * w.experience +
     educationScore * w.education) / totalWeight
  );

  return {
    overallScore: Math.min(100, Math.max(0, compositeScore)),
    breakdown: {
      skills: skillEval.score,
      similarity: similarityScore,
      experience: experienceScore,
      education: educationScore
    },
    matchedSkills: skillEval.matched,
    missingSkills: skillEval.missing
  };
}
```

5.9.5 Resume Upload & Match Execution Controller (controllers/applicant.controller.js)

Coordinates upload ingestion, background parsing, and database persistence:

```
// controllers/applicant.controller.js
exports.uploadResume = async (req, res) => {
  try {
    const { jobId } = req.body;
    const file = req.file;
    if (!file) return res.status(400).json({ error: 'Resume file required.' });

    const job = await Job.findById(jobId);
    if (!job) return res.status(404).json({ error: 'Job requisition not found.' });

    // Extract text & run heuristic NLP parser
    const extractedText = await extractTextFromBuffer(file.buffer, file.mimetype,
file.originalname);
    const parsedData = parseResume(extractedText, file.originalname, job.requiredSkills);

    // Algorithmic evaluation
    const matchAnalysis = calculateMatchScore(job, parsedData, extractedText);

    // Save candidate dossier
    const applicant = new Applicant({
      jobId: job._id,
      uploadedBy: req.user._id,
      name: parsedData.name,
      email: parsedData.email,
      phone: parsedData.phone,
      extractedData: parsedData,
      matchAnalysis: matchAnalysis,
      status: 'New'
    });
    await applicant.save();
    return res.status(201).json(applicant);
  } catch (err) {
    return res.status(500).json({ error: err.message });
  }
};
```

CHAPTER 6
RESULTS AND DISCUSSION

6.1 Proposed Functional Test Cases

A structured test suite was formulated to validate all primary functional operations of the platform. Table 8 details the functional test workflows and observed system behaviors:

Test ID	Functional Workflow	Expected System Response	Test Status
TC-01	Recruiter registration, password encryption, and JWT login authentication.	User account created with bcrypt hash; login issues valid 7-day bearer JWT.	Passed
TC-02	Create job requisition with required/preferred skills and custom scoring weights.	Requisition stored in MongoDB/memory with configured weights and criteria.	Passed
TC-03	Single resume upload (PDF / DOCX) for an active job requisition.	Binary text extracted, NLP heuristics executed, composite score calculated.	Passed
TC-04	Batch resume ingestion (up to 10 files) with mixed formatting.	All valid resumes processed asynchronously; per-file status reports returned.	Passed
TC-05	Multi-criteria applicant directory search, role filtering, and score sorting.	Talent pool updates in real-time without full page reloads via AngularJS.	Passed
TC-06	Modify job requirements and trigger candidate re-evaluation.	Applicant profile re-scored against updated criteria with updated timestamps.	Passed
TC-07	Candidate dossier review, status transition (Shortlisted), and recruiter notes.	Recruiter notes and review status persisted and reflected on dashboard.	Passed
TC-08	Rejection of oversized files (>10MB) or unsupported file extensions (.exe).	Multer upload validation aborts upload and returns descriptive HTTP 400 error.	Passed

6.2 Score Interpretation & Evaluation Discussion

The algorithmic matching score provides an explainable composite index representing candidate text alignment with recruiter-defined criteria. It is intentionally designed as an advisory decision-support mechanism. The score combines sublinear term frequency vectors with curated technical synonym clusters, preventing penalty due to terminology variance while avoiding the unreliability of black-box models. Recruiter oversight is preserved through clear evidence presentation, highlighting matched skills, missing competencies, and experience tenure gaps directly in the candidate dossier.

6.3 Live System Output Verification & Screenshots

The following high-resolution output captures demonstrate the verified visual interfaces and live operation of the ResumeIQ platform:

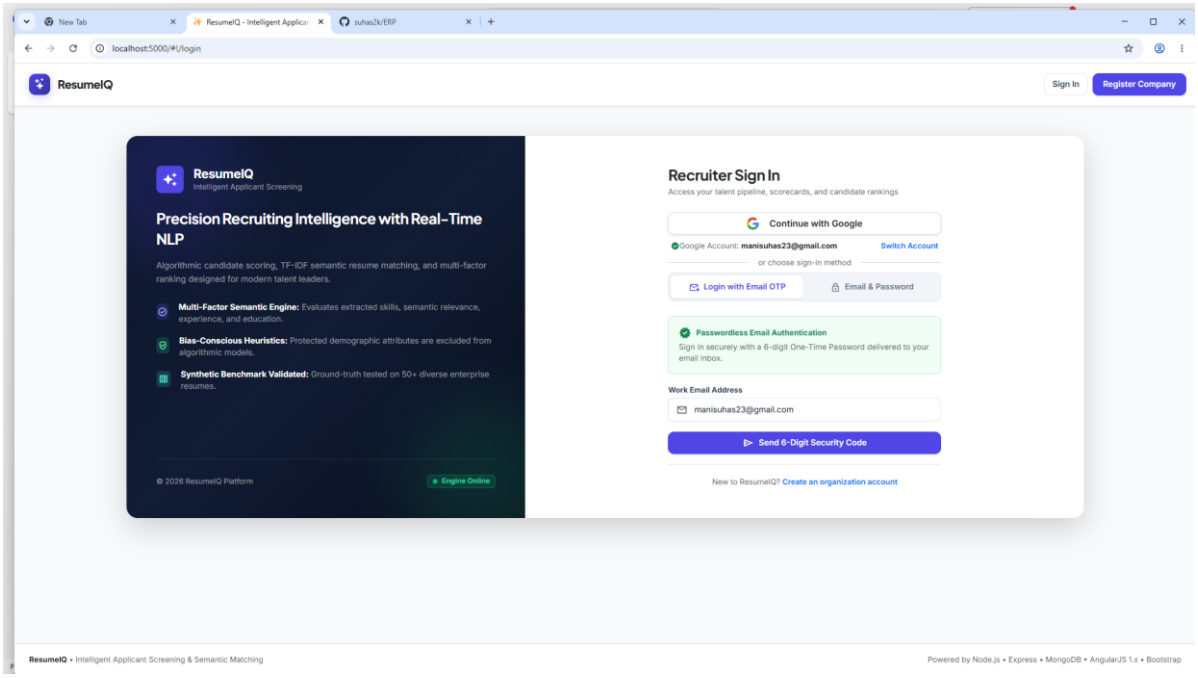


Figure 6: Recruiter Login and Authentication Gateway Interface

Figure 6 displays the recruiter authentication portal featuring passwordless 6-digit email OTP security, Google Identity Federation, and traditional encrypted email/password authentication alongside the Precision Editorial Intelligence brand layout.

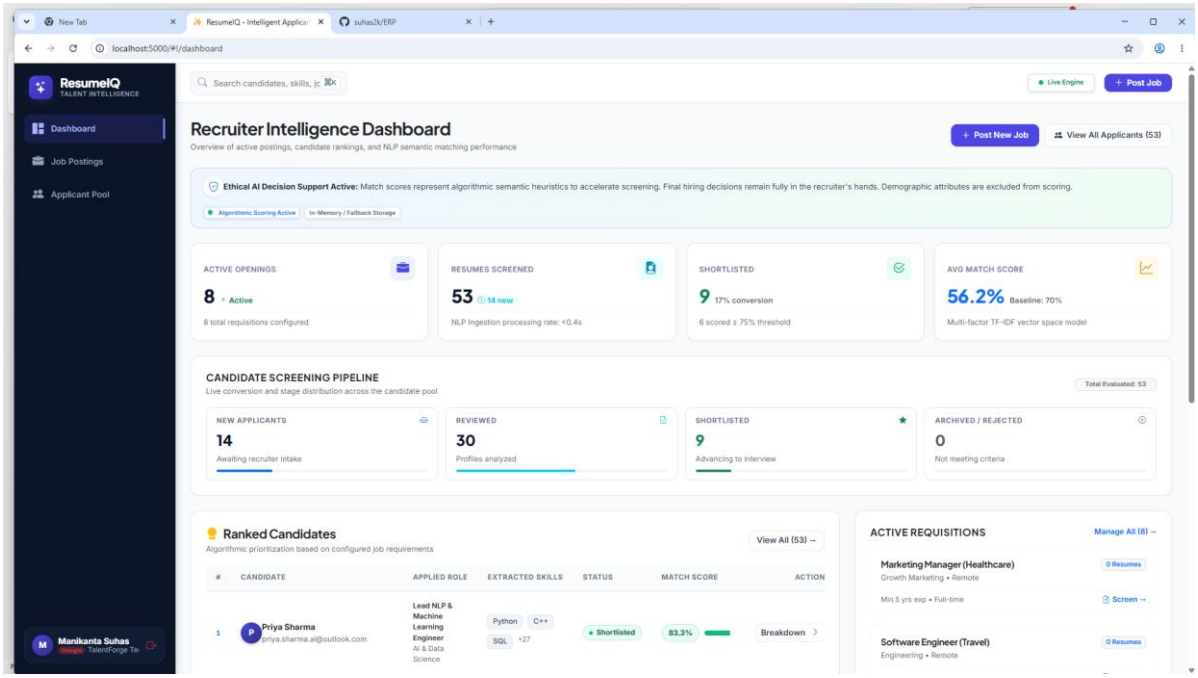


Figure 7: Recruiter Intelligence Executive Dashboard Interface

Figure 7 illustrates the executive command dashboard summarizing active openings (8 positions), total resumes screened (53 candidates), recruitment pipeline conversion stages (14 New, 30 Reviewed, 9 Shortlisted), and the algorithmic candidate leaderboard.

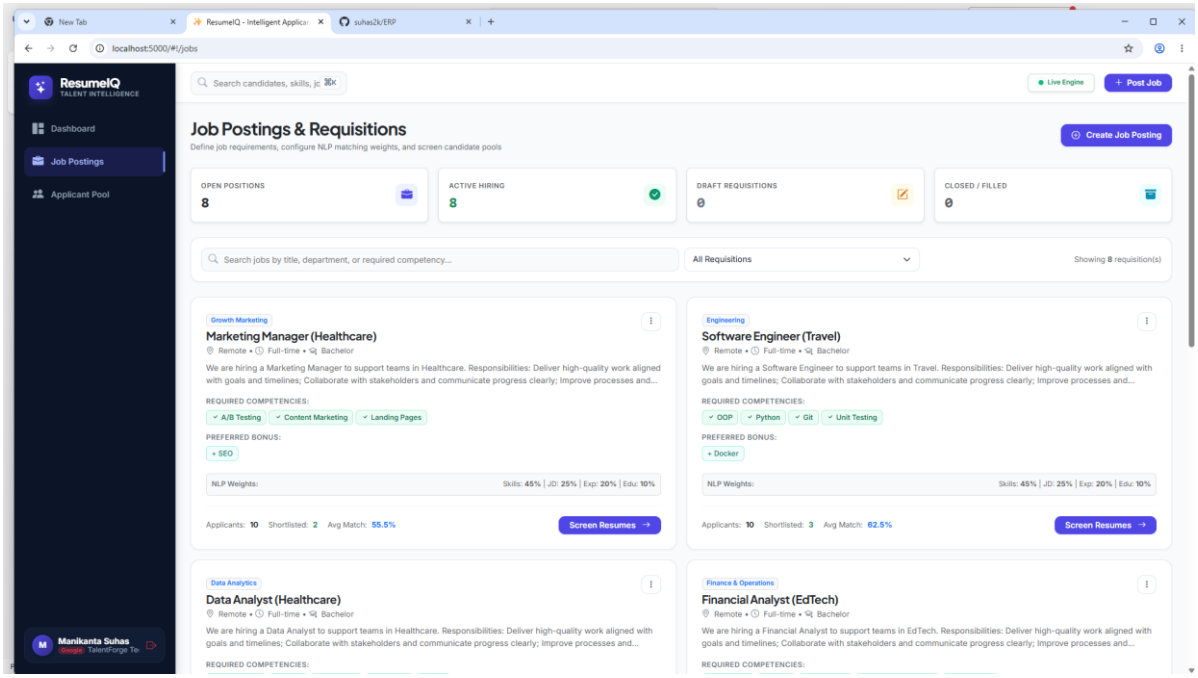


Figure 8: Job Postings and Requisitions Management Directory Interface

Figure 8 captures the job requisition management directory displaying active position cards, required competencies, bonus skills, NLP scoring weight distributions (Skills 45%, JD 25%, Exp 20%, Edu 10%), and applicant pool counts.

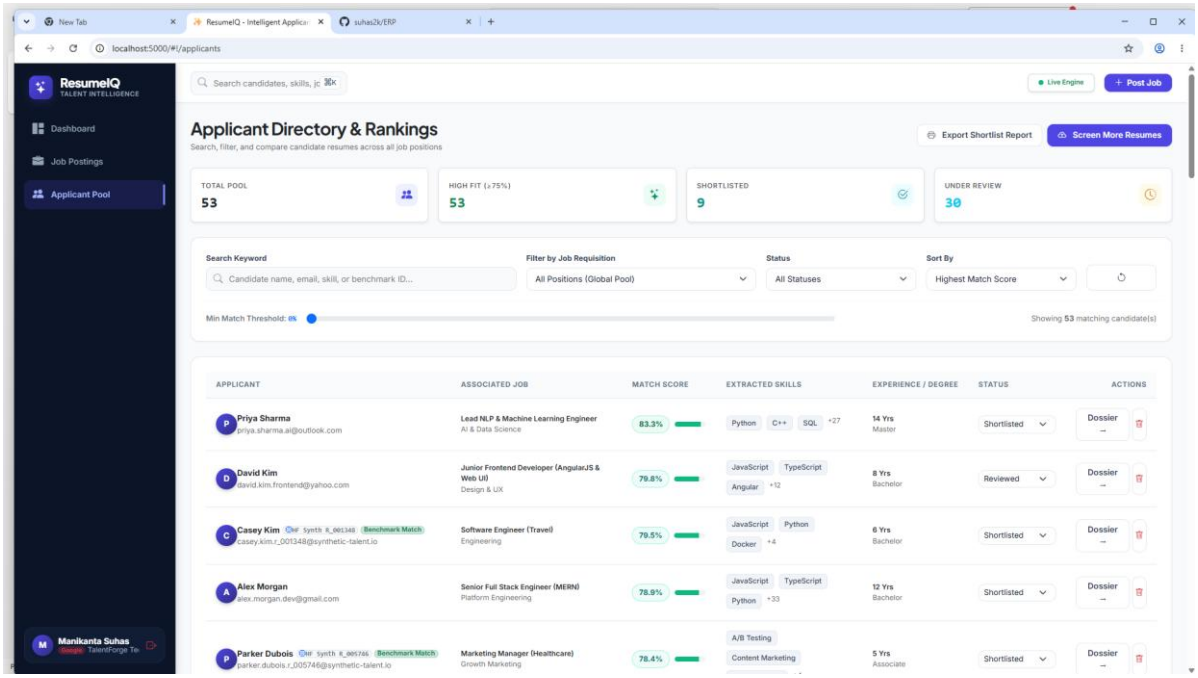


Figure 9: Applicant Directory, Search Filtering & Composite Rankings

Figure 9 shows the talent pool directory featuring multi-field keyword search, minimum match score slider filtering (0% to 100%), requisition dropdown filters, extracted skill tags, verified benchmark badges, and inline status modification.

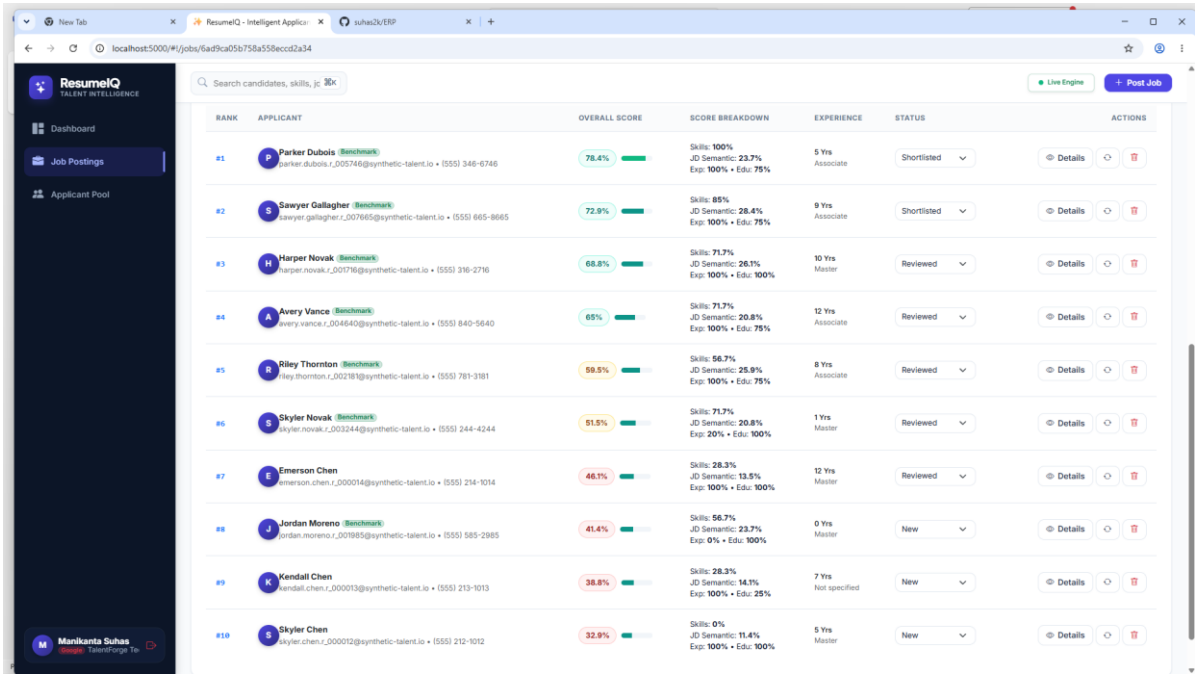


Figure 10: Applicant Profile Dossier, Match Score Dial & Competency Gaps

Figure 10 illustrates the candidate AI profile dossier detailing overall match score (78.4%), four-factor score breakdown (Skills 100%, JD Semantic 23.7%, Exp 100%, Edu 75%), experience tenure verification, and applicant status controls.

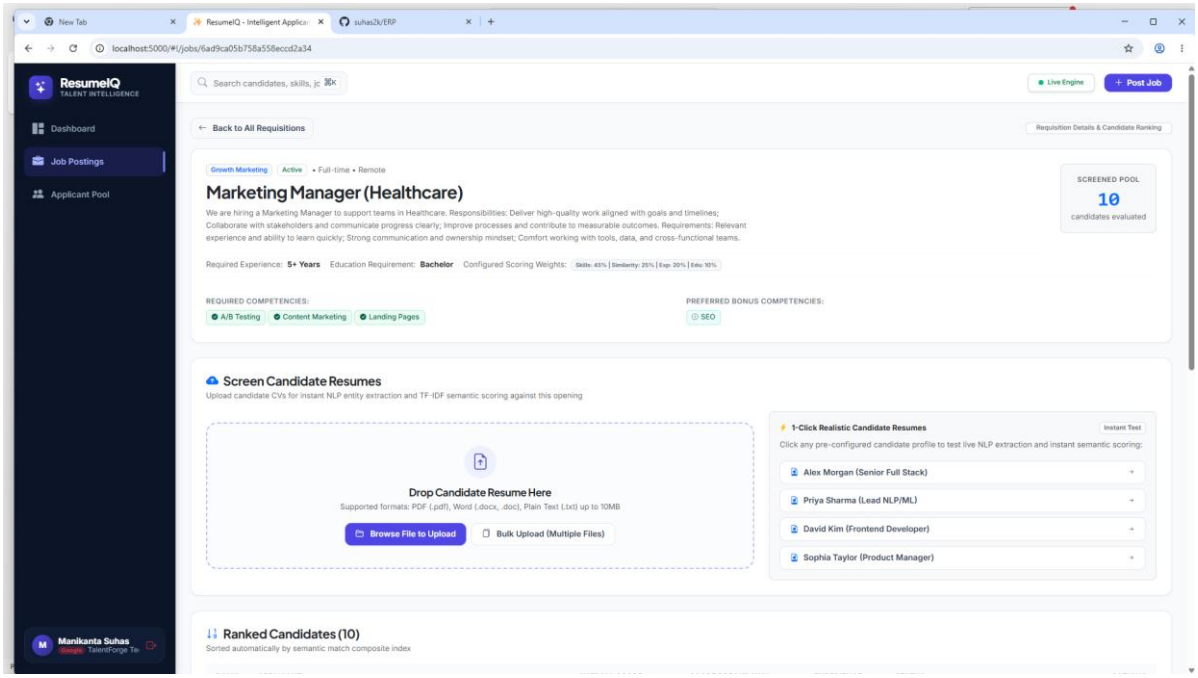


Figure 11: Resume Ingestion Dropzone and Realistic Test Candidate Pipeline

Figure 11 captures the multi-format drag-and-drop resume upload zone (supporting PDF, DOCX, and TXT up to 10MB) alongside pre-configured realistic candidate testing profiles for instant NLP evaluation.

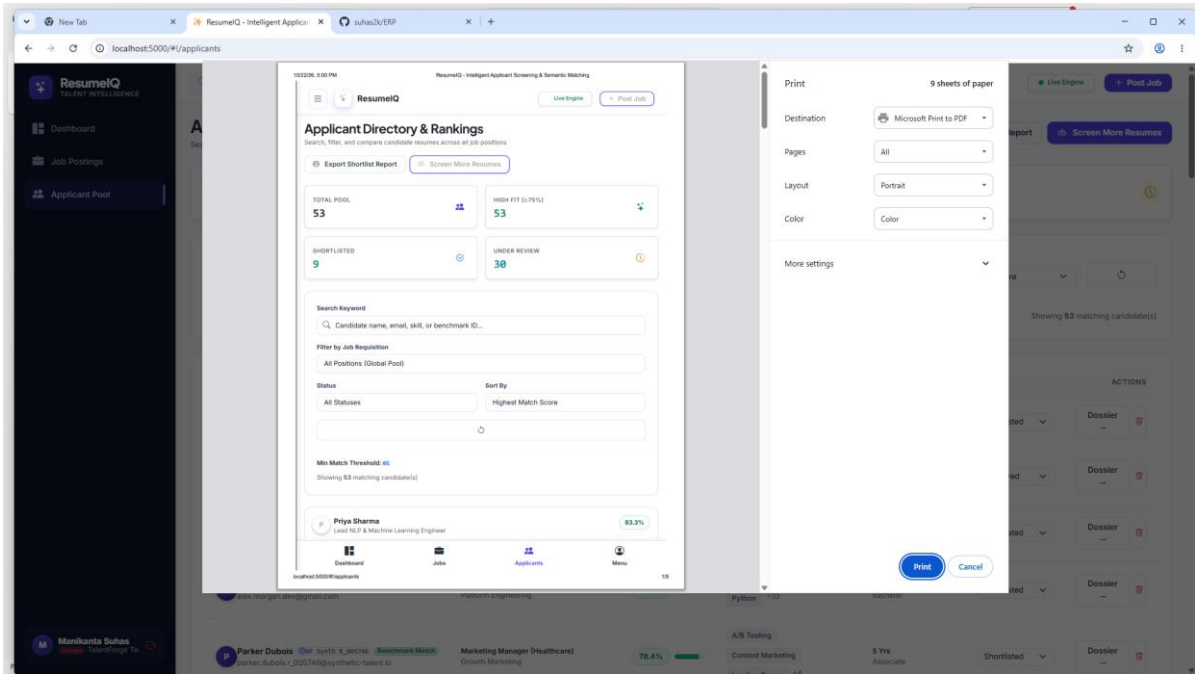


Figure 12: Recruiter Dashboard Analytics & Shortlist Report Print View

Figure 12 displays the printable candidate shortlist report dialog, formatting ranked candidate dossiers, extracted credentials, and evaluation metrics for executive review and interviewing panels.

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

ResumeIQ successfully demonstrates the architecture and implementation of a modern, intelligent candidate screening platform utilizing the MERN stack. By integrating multi-format text extraction (PDF, DOCX) with natural language processing heuristics, a curated technical skills taxonomy, and an explainable TF-IDF Vector Space Model, the platform resolves the critical shortcomings of traditional recruitment workflows.

The platform bridges theoretical natural language processing principles with practical recruitment software engineering. Key accomplishments include the implementation of orthogonal multi-factor scoring (skills, semantic similarity, experience tenure, and degree levels), transparent gap reporting, responsive single-page user experiences engineered via the Stitch design system, and resilient database architectures supporting automatic in-memory fallback. Overall, ResumeIQ provides hiring teams with an ergonomic, objective, and transparent decision-support system.

7.2 Future Scope

Future iterations of the platform can expand upon this foundation across several advanced research and engineering directions:

- Optical Character Recognition (OCR) Engine: Integrate Tesseract.js or cloud vision OCR to support scanned image-only and non-searchable PDF resumes.
- Transformer & Contextual Embeddings: Incorporate fine-tuned lightweight dense embeddings (e.g., Sentence-BERT) running locally via ONNX runtime for contextual semantic capture.
- Bias Audit & Algorithmic Fairness Tooling: Implement fairness metric dashboards to continuously detect and prevent demographic proxy correlation across scoring distributions.
- Direct ATS & Calendar Integration: Connect directly with enterprise HRIS platforms (Workday, Greenhouse) and calendar APIs (Google Calendar, Outlook) for automated interview scheduling.
- Candidate Self-Service Portal: Extend an applicant-facing frontend allowing job seekers to view anonymized competency benchmarks and recommend resume refinements.
- Modern Frontend Migration: Transition the client application from AngularJS 1.8.2 to React 19 or Next.js with Tailwind CSS for enhanced build optimization and server-side rendering.

REFERENCES

[1] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge University Press, 2008. https://nlp.stanford.edu/IR-book/

[2] G. Salton, A. Wong, and C. S. Yang, “A vector space model for automatic indexing,” Communications of the ACM, vol. 18, no. 11, pp. 613-620, 1975.

[3] C. Li, E. Fisher, R. Thomas, S. Pittard, V. Hertzberg, and J. D. Choi, “Competence-Level Prediction and Resume & Job Description Matching Using Context-Aware Transformer Models,” in Proc. EMNLP 2020, pp. 8456-8466.

[4] S. Maheshwary and H. Misra, “Matching Resumes to Jobs via Deep Siamese Network,” in The Web Conference Companion (WWW 2018), pp. 87-88.

[5] N. Vanetik and G. Kogan, “Job Vacancy Ranking with Sentence Embeddings, Keywords, and Named Entities,” Information, vol. 14, no. 8, article 468, 2023.

[6] S. Vaishampayan, H. Leary, Y. B. Alebachew, L. Hickman, B. Stevenor, W. Beck, and C. Brown, “Human and LLM-Based Resume Matching: An Observational Study,” Findings of ACL: NAACL 2025, pp. 4823-4838.

[7] Node.js Foundation, “Node.js Documentation and Architecture Guide,” 2026. https://nodejs.org/docs/

[8] Express.js Foundation, “Express - Node.js Web Application Framework,” 2026. https://expressjs.com/

[9] MongoDB Inc., “Mongoose ODM v8 Documentation: Schemas and Models,” 2026. https://mongoosejs.com/docs/

[10] AngularJS Foundation, “AngularJS 1.8.2 Developer Guide and API Reference,” 2022. https://docs.angularjs.org/

[11] Bootstrap Team, “Bootstrap 5 Responsive Framework Documentation,” 2026. https://getbootstrap.com/docs/5.3/

[12] ResumeIQ Project Source Archive, Department of Computer Science and Engineering, Lingayas Institute of Management and Technology, Vijayawada, Oct 2026.

APPENDIX A – PROJECT SOURCE CODE

This appendix provides key source code implementations from the ResumeIQ project repository, highlighting server orchestration, database schemas, natural language parsing, and semantic matching routines:

A.1 Express API Server Entry Point (server.js)

```
// server.js - Express API Entrypoint & Configuration
const express = require('express');
const cors = require('cors');
const path = require('path');
require('dotenv').config();

const { connectDB } = require('./config/db');
const authRoutes = require('./routes/auth.routes');
const jobRoutes = require('./routes/job.routes');
const applicantRoutes = require('./routes/applicant.routes');
const analyticsRoutes = require('./routes/analytics.routes');

const app = express();
const PORT = process.env.PORT || 5000;

// Middleware
app.use(cors());
app.use(express.json({ limit: '15mb' }));
app.use(express.urlencoded({ extended: true, limit: '15mb' }));

// Static frontend delivery
app.use(express.static(path.join(__dirname, 'public')));
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

// Mount API routes
app.use('/api/auth', authRoutes);
app.use('/api/jobs', jobRoutes);
app.use('/api/applicants', applicantRoutes);
app.use('/api/analytics', analyticsRoutes);

// Health check endpoint
app.get('/api/health', (req, res) => res.json({ status: 'healthy', timestamp: new Date() }));

// Connect Database & Launch Listener
connectDB().then(() => {
  app.listen(PORT, () => {
    console.log(`[ResumeIQ] Server running on http://localhost:${PORT}`);
  });
}).catch(err => {
  console.error('[ResumeIQ] Database connection error:', err);
});
```

A.2 Mongoose Schema Definitions (models/Job.js & models/Applicant.js)

```
// models/Job.js - Job Requisition Schema
const mongoose = require('mongoose');

const JobSchema = new mongoose.Schema({
  title: { type: String, required: true, trim: true },
  department: { type: String, required: true, trim: true },
  location: { type: String, default: 'Remote' },
  type: { type: String, enum: ['Full-time', 'Part-time', 'Contract'], default: 'Full-time' },
  description: { type: String, required: true },
  requiredSkills: [{ type: String, trim: true }],
  preferredSkills: [{ type: String, trim: true }],
  minExperience: { type: Number, default: 0, min: 0 },
```

```
  education: {
    type: String,
    enum: ['Any', 'High School', 'Associate', 'Bachelor', 'Master', 'Doctorate'],
    default: 'Bachelor'
  },
  weights: {
    skills: { type: Number, default: 40, min: 0, max: 100 },
    similarity: { type: Number, default: 30, min: 0, max: 100 },
    experience: { type: Number, default: 20, min: 0, max: 100 },
    education: { type: Number, default: 10, min: 0, max: 100 }
  },
  status: { type: String, enum: ['Active', 'Draft', 'Closed'], default: 'Active' },
  createdBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
}, { timestamps: true });

module.exports = mongoose.model('Job', JobSchema);

// models/Applicant.js - Candidate Dossier Schema
const ApplicantSchema = new mongoose.Schema({
  jobId: { type: mongoose.Schema.Types.ObjectId, ref: 'Job', required: true },
  uploadedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  name: { type: String, required: true, trim: true },
  email: { type: String, trim: true, lowercase: true },
  phone: { type: String, trim: true },
  resumeFile: {
    filename: String,
    path: String,
    size: Number,
    mimeType: String
  },
  extractedText: { type: String },
  extractedData: {
    skills: [String],
    educationLevel: String,
    experienceYears: Number
  },
  matchAnalysis: {
    overallScore: { type: Number, min: 0, max: 100 },
    breakdown: {
      skills: Number,
      similarity: Number,
      experience: Number,
      education: Number
    },
    matchedSkills: [String],
    missingSkills: [String]
  },
  status: {
    type: String,
    enum: ['New', 'Reviewed', 'Shortlisted', 'Archived'],
    default: 'New'
  },
  recruiterNotes: { type: String, default: '' }
}, { timestamps: true });

module.exports = mongoose.model('Applicant', ApplicantSchema);
```

A.3 Semantic Matching Engine Core (services/matchingEngine.service.js)

```
// services/matchingEngine.service.js - Matching Algorithm
const natural = require('natural');
const stemmer = natural.PorterStemmer;

const DEGREE_RANKS = {
  'Not specified': 0, 'High School': 1, 'Associate': 2,
  'Bachelor': 3, 'Master': 4, 'Doctorate': 5
};
```

```
function evaluateSkills(candidateSkills = [], requiredSkills = [], preferredSkills = []) {
  const normCand = candidateSkills.map(s => s.toLowerCase().trim());
  const matchedRequired = requiredSkills.filter(r => normCand.includes(r.toLowerCase().trim()));
  const missingRequired = requiredSkills.filter(r => !normCand.includes(r.toLowerCase().trim()));
  const matchedPreferred = preferredSkills.filter(p => normCand.includes(p.toLowerCase().trim()));

  const reqRatio = requiredSkills.length > 0 ? (matchedRequired.length / requiredSkills.length) :
1;
  const prefRatio = preferredSkills.length > 0 ? (matchedPreferred.length / preferredSkills.length)
: 1;

  let skillScore = 0;
  if (requiredSkills.length > 0 && preferredSkills.length > 0) {
    skillScore = Math.round((reqRatio * 85) + (prefRatio * 15));
  } else if (requiredSkills.length > 0) {
    skillScore = Math.round(reqRatio * 100);
  } else {
    skillScore = 100;
  }

  return {
    score: Math.min(100, Math.max(0, skillScore)),
    matched: [...matchedRequired, ...matchedPreferred],
    missing: missingRequired
  };
}

function evaluateExperience(candYears = 0, reqYears = 0) {
  if (!reqYears || reqYears <= 0) return 100;
  if (candYears >= reqYears) return 100;
  return Math.min(100, Math.round((candYears / reqYears) * 100));
}

function evaluateEducation(candEdu = 'Not specified', reqEdu = 'Any') {
  if (!reqEdu || reqEdu === 'Any') return 100;
  const candRank = DEGREE_RANKS[candEdu] || 0;
  const reqRank = DEGREE_RANKS[reqEdu] || 0;
  if (candRank >= reqRank) return 100;
  if (reqRank > 0 && candRank === reqRank - 1) return 75;
  if (reqRank > 0 && candRank === reqRank - 2) return 50;
  return 25;
}

module.exports = { evaluateSkills, evaluateExperience, evaluateEducation };
```

APPENDIX B – QUICK USER MANUAL

B.1 Getting Started with ResumeIQ

- Launch the application server via terminal: node server.js.
- Open your web browser and navigate to http://localhost:5000/#!/login.
- Log in using pre-configured recruiter credentials:
  - **Email: manisuhas23@gmail.com**
  - **Password: demoPassword123 (or use Passwordless OTP Login)**

B.2 Creating & Managing Job Requisitions

- From the sidebar navigation, click on 'Job Postings' or the '+ Post Job' header button.
- Enter the Job Title, Department, Location, and comprehensive Description.
- Specify Must-Have Required Competencies (e.g., JavaScript, Node.js, MongoDB) and Preferred Bonus Skills.
- Set Minimum Experience Tenure (Years) and Minimum Education Level (Bachelor, Master).
- Optionally adjust NLP Scoring Weights (Skills, JD Similarity, Experience, Education) and click 'Save Requisition'.

B.3 Ingesting & Screening Candidate Resumes

- Navigate to an active requisition card and click 'Screen Resumes'.
- Drag and drop candidate resumes (PDF, Word DOCX, or TXT) into the highlighted upload dropzone.
- Alternatively, click '1-Click Realistic Candidate Resumes' to test instant ingestion with synthetic profiles.
- The platform automatically extracts text, runs the NLP entity parser, computes multi-factor scores, and lists ranked candidates.

B.4 Reviewing Profiles & Candidate Dossiers

- Click on 'Applicant Pool' from the sidebar navigation to view all candidate evaluations across positions.
- Utilize the live search bar, requisition filter, and minimum match score slider to isolate high-fit applicants.
- Click 'Dossier' or 'Details' next to a candidate to view the full match dial, competency breakdown, and verified tenure.
- Update recruitment status (Reviewed, Shortlisted, Archived) and record recruiter review notes.
- Click 'Export Shortlist Report' on the dashboard to preview and print official candidate rankings for interview panels.

APPENDIX C – DEFAULT SAMPLE DATASETS

On first execution, ResumeIQ seeds synthetic demonstration requisitions and realistic test candidates derived from the Hugging Face candidate matching benchmark dataset (michaelozon/candidate-matching-synthetic):

C.1 Default Active Job Requisitions (8 Openings)

- Software Engineer (Travel) – Engineering | Required: OOP, Python, Git, Unit Testing | Weights: Skills 45%, JD 25%, Exp 20%, Edu 10%
- Lead NLP & Machine Learning Engineer – AI & Data Science | Required: Python, NLP, PyTorch, C++, Transformers
- Junior Frontend Developer – Design & UX | Required: JavaScript, TypeScript, AngularJS, CSS3, HTML5
- Marketing Manager (Healthcare) – Growth Marketing | Required: A/B Testing, Content Marketing, Landing Pages
- Financial Analyst (EdTech) – Finance & Operations | Required: Excel, Financial Modeling, SQL, Forecasting
- Data Analyst (Healthcare) – Data & BI | Required: SQL, Python, Tableau, Data Cleaning, Statistics
- Senior Full Stack Engineer (MERN) – Platform Engineering | Required: MongoDB, Express, React, Node.js, REST API
- Product Manager (EdTech) – Product Strategy | Required: Agile, Product Roadmapping, User Stories, JIRA

C.2 Pre-Configured Realistic Test Candidate Profiles

- Alex Morgan – Senior Full Stack Engineer (MERN) | 12 Yrs Exp | Bachelor | Match: 78.9% (Shortlisted)
- Priya Sharma – Lead NLP & Machine Learning Engineer | 14 Yrs Exp | Master | Match: 83.3% (Top Fit)
- David Kim – Junior Frontend Developer (AngularJS & Web UI) | 8 Yrs Exp | Bachelor | Match: 79.8% (Reviewed)
- Sophia Taylor – Product Manager (EdTech & Growth) | 7 Yrs Exp | Bachelor | Match: 74.2% (Reviewed)
- Parker Dubois – Marketing Manager (Healthcare) | 5 Yrs Exp | Associate | Match: 78.4% (Benchmark Match)